

두 가지 기호 실행 기술들의 융합하는 방법들

김문수⁰, 차수영

성균관대학교 소프트웨어학과

kimmail99@g.skku.edu, sooyoung.cha@skku.edu

Methods for Combining Two Symbolic Execution Techniques¹⁾

Moonsu Kim⁰, Sooyouncha Cha

Department of Computer Science and Engineering, Sungkyunkwan University

요 약

기호 실행(Symbolic Execution)은 잘 알려진 하나의 소프트웨어 테스트 방법론이다. 기호 실행은 프로그램의 실행 경로들을 체계적으로 탐색하여 높은 코드 커버리지 및 다양한 오류 탐지를 목표로 한다. 하지만 실제 프로그램을 테스트할 때, 기호 실행은 여전히 경로-폭발 문제, 높은 제약식 풀이 비용 등 도전적인 과제 때문에 더 높은 테스트 성능을 성취하지 못하고 있다. 이를 완화하기 위해, 기호 실행 연구자들은 탐색 전략, 공간 축소 전략, 파라미터 조정 전략 등 다양한 개별적인 기술들을 고안하였다. 그러나 다양한 기호 실행 기술들을 동시에 사용하는 방법론은 연구가 되지 않았다. 본 논문은 두 가지 대표적인 기호 실행 기술들을 동시에 사용하는 방법론 및 그 가능성에 대해 모색한다. 구체적으로, 본 연구는 기호 실행의 최신 기술 두 가지 “탐색 전략”과 “외부 파라미터 조정 전략”을 융합하는 방법들과 그 융합의 효과성을 확인해 보았다. 본 논문의 실험 결과는 탐색 전략과 외부 파라미터 조정 전략을 개별적으로 사용하는 경우보다 두 가지를 융합한 방식이 코드 커버리지 측면에서 평균적으로 가장 우수한 성능을 보였고 새로운 버그를 효과적으로 검출하는 데에도 기여함을 보였다.

1. 서 론

현대 소프트웨어의 복잡성 증가에 따라, 시스템 결함의 위험도 함께 커지고 있다. 이를 완화하기 위한 대표적 테스트 기법인 기호 실행(Symbolic Execution)은 입력을 기호 변수로 치환하고 SMT(Satisfiability Modulo Theories) 풀이기로 분기 조건을 분석해 다양한 실행 경로를 탐색한다. 하나의 잘 알려진 기호 실행 도구로는 KLEE [1]가 있다. KLEE는 실제 C 프로그램을 대상으로 테스트를 수행하며 주어진 시간 동안 높은 코드 커버리지를 달성하는 기호 실행 도구이다. 그러나 KLEE와 같은 기호 실행 도구들은 대규모 시스템을 대상으로 할 때, 경로 수가 기하급수적으로 증가하는 경로 폭발(Path Explosion) 문제와 높은 제약식 풀이 비용(Expensive Constraint Solving)은 여전히 해결되지 않은 주요 과제이다.

이러한 한계를 완화하기 위해 학계에서는 다양한 보완 기법을 제안해 왔다. 첫째, 우선 탐색할 상태를 선택해 불필요한 분기 확장을 억제하는 탐색 전략(Search Strategies) [2]. 둘째, SMT 타임아웃이나 메모리 한도 같은 실행

환경 파라미터를 동적으로 조정해 제약식 풀이 비용을 줄이는 외부 파라미터 조정(Parameter Tuning) [3] 기법이 있다. 이 밖에도, 제약식을 단순화하거나 분할·정복 기법을 적용하는 SMT 풀이 최적화 기법(Solver Optimizations) [4], 의미 없는 경로를 사전에 제거하는 Path-Pruning [5], 유사 상태를 병합해 탐색 공간을 축소하는 State-Merge [6] 기법 등이 보고되었다.

그러나 이들 개별 기법은 각자 독립적인 관점에서 성능을 개선했을 뿐, 서로 다른 기법을 체계적으로 결합하는 연구는 거의 이루어지지 않았다. 예컨대 탐색 전략과 외부 파라미터 조정 전략처럼 서로 다른 방법론 간의 성능을 직접 비교하거나, 이질적인 전략들을 결합하여 융합 효과를 분석한 연구는 거의 이루어지지 않았다. 특히, 이들 전략을 함께 적용했을 때의 시너지나 상호보완적 가능성을 실험적으로 검증하려는 시도는 매우 드물며, 두 기법의 결합 방식에 따른 성능 차이를 체계적으로 비교한 연구는 전무한 실정이다.

본 논문은 두 가지 최신 기호 실행 기술인 탐색 전략 [2] 과 외부 파라미터 조정 전략[3]의 결합을 세 가지 방법으로 시도하여 융합 가능성을 탐구한다. 첫째, 동시에 실행하며 상호 피드백을 주고받는 병렬 융합, 둘째, 전체 실행 시간을 동등하게 분할해 적용하는 균형 융합, 셋째, 두 전략을 주기적으로 교대로 적용하는 교대 융합이다.

본 논문은 5개의 GNU C 벤치마크들에 대해서 10시간 동안 각 방법론들의 성능을 평가하였다. 실험 결과 교대

1) 이 논문은 2025년도 정부(과학기술정보통신부)의 재원으로 정보통신기획평가원의 지원(RS-2019-II190421, 인공지능대학원지원(성균관대학교))과 2025년도 정부(과학기술정보통신부)의 재원으로 정보통신기획평가원의 지원을 받아 수행된 연구 결과임(No.RS-2024-00438686, 비정상 오픈소스 식별 및 DevSecOps 자동 적용을 통한 소프트웨어 신뢰성 향상 기술 개발).

융합은 단일 전략 대비 코드 커버리지 성능에서 가장 뛰어났으며, 병렬 융합은 기존 모든 방법이 놓친 결함을 새롭게 검출하였다. 이를 통해 본 논문은 서로 다른 기법의 융합 가능성에 대한 예비적 증거를 제공한다.

2. 융합 시도 전략

다음은 각 융합 시도의 구조와 공유 메커니즘에 대한 설명이다.

병렬 융합(Parallel Fusion)은 탐색 전략과 외부 파라미터 조정 전략이 병렬로 실행된다. KLEE 기호 실행의 파라미터와 탐색 전략은 각각 직전 실행에서의 탐색 전략, 외부 파라미터 조정 전략의 업데이트 함수의 출력을 참조하여 학습한다. 탐색 전략의 업데이트 함수는 현재 파라미터 설정 하에 생성된 경로(상태) 샘플에 대해, 각 경로의 특성과 가중치를 통해 우선순위를 계산한다.

한편, 외부 파라미터 조정 전략 업데이트 함수는 고정된 상태 전략과 함께 여러 파라미터 조합을 시험 실행한다. 이후 코드 커버리지, 실패율 그리고 가중치를 사용하여 성능을 평가하여 높은 우선순위의 파라미터들을 생성한다.

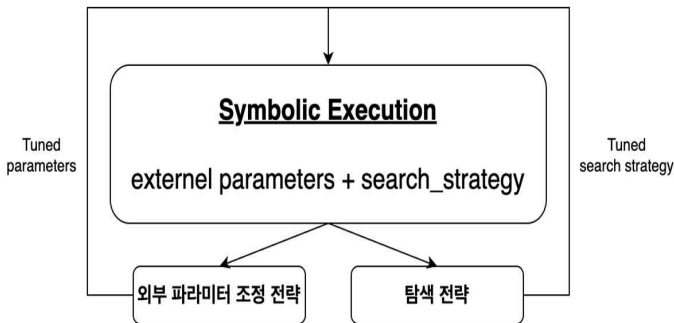


그림 1. 병렬 융합 모델

균형 융합은 전체 실행 시간을 절반으로 나누어, 전반부는 외부 파라미터 조정 전략을, 후반부는 기반 탐색을 수행하는 구조이다. 전반부 과정에서 탐색 성능이 가장 높은 파라미터를 학습하고, 후반부에는 선정된 파라미터를 고정하여 KLEE를 실행하며, 탐색 전략을 갱신한다.

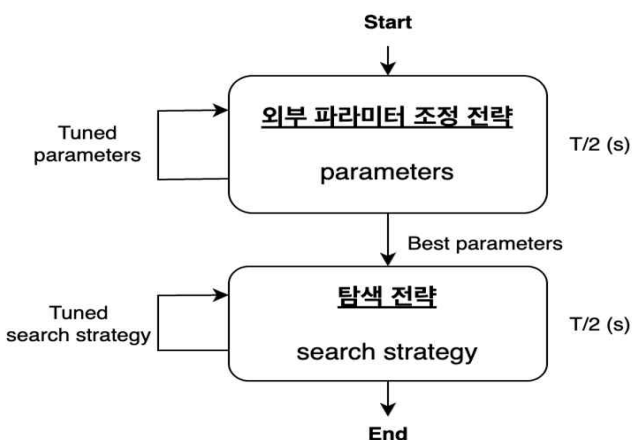


그림 2. 균형 융합 모델

교대 융합은 고정된 시간 간격을 기준으로 탐색 전략과 외부 파라미터 조정 전략을 교대로 실행하며, 각 주기에서 얻은 결과를 다음 모듈에 피드백하는 구조이다. 탐색 전략은 직전의 외부 파라미터 조정 전략에서 제시한 파라미터 설정 하에서 갱신된다. 학습된 탐색 전략은 다음 주기에서 외부 파라미터 조정 전략이 우선 조정해야 할 경로의 힌트로 활용된다.

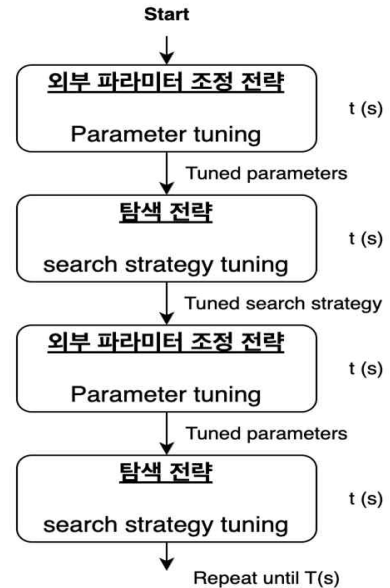


그림 3. 교대 융합 모델

3. 실험 결과

본 연구는 새로 제안한 세 가지 결합 모델(병렬, 균형, 교대)을 기존 자동화 기법(탐색 전략, 외부 파라미터 조정 전략)과 베이스라인 도구 KLEE에 대해 비교·평가하였다. 실험에는 심볼릭 실행 성능 측정에 널리 사용되는 다섯 개의 GNU 오픈소스 C 프로그램(combine-0.4.0, diffutils-3.7, gcal-4.1, ls- 8.32, trueprint-5.4)을 벤치마크로 활용하였다. 각 벤치마크는 동일한 자원 환경에서 36000초 동안 기호 실행을 수행했으며, 분기 커버리지(Branch Coverage)와 버그 검출 능력을 성능 지표로 삼았다.

전체 벤치마크 평균으로 볼 때, 교대 융합이 가장 높은 커버리지를 달성하여 KLEE 대비 131.8%, 탐색 전략 대비 15.8%, 외부 파라미터 조정 전략 대비 13.6% 더 많은 분기를 탐색했다. 균형 융합은 탐색 전략보다 3.5%, 외부 파라미터 조정 전략보다 1.5% 높은 성능을 보였다. 병렬 융합은 KLEE 대비 98.3% 더 많은 분기를 탐색했으나, 탐색·파라미터 조정 전략보다는 각각 0.9%, 2.9% 낮은 평균 커버리지를 기록했다.

벤치마크별로 살펴보면, 교대 융합은 diffutils, gcal, ls에서 최고 커버리지를 달성했다. 특히, gcal-4.1에서는 4,164개의 분기를 탐색해 KLEE(2,017개)의 두 배를 넘었고, 탐색 전략 대비 33.5% 우수한 성과를 보였다. trueprint-5.4에서는 균형 융합이 1,017개의 분기

Benchmark	탐색 전략[2]	파라미터 조정 전략[3]	KLEE[1]	병렬 융합	균형 융합	교대 융합
combine-0.4.0	930	934	403	1190	801	1160
diffutils-3.7	2217	1780	732	1820	1947	2401
gcal-4.1	3120	3930	2017	3300	3801	4164
ls-8.32	1802	1702	901	1801	1751	1897
trueprint-5.4	934	834	445	807	1017	806
합 계	9003	9180	4498	8918	9317	10428

표 1. 5개 벤치마크에서 각 기법이 36000초 동안 달성한 최고 분기 커버리지

Error-types	error location	탐색 전략	파라미터 조정 전략	KLEE	병렬 융합	균형 융합	교대 융합
Segmentation fault	Line:385 in /src/field.c	✓	✓	✓	✓	✓	✓
mem-exhaustion	Line:48 in /string/memmove.c	✗	✓	✗	✗	✗	✗
Segmentation fault	Line:815 in /src/dstring.c	✗	✗	✗	✓	✗	✗

표 2. combine-0.4.0 벤치마크에서 각 기법이 36000초 동안 발견한 버그들

를 탐색하며 최고 성능을 나타내, 탐색 전략과 외부 파라미터 조정 전략을 각각 8.9%, 21.9% 상회했다. combine-0.4.0에서는 병렬 융합이 1,190개의 분기를 탐색해 가장 높은 커버리지를 기록했다.

버그 검출 측면에서도 특정 융합 기법의 우수성을 관찰할 수 있었다. 병렬 융합은 combine-0.4.0에서 기존 모든 기법이 놓친 dstring.c:815 경로의 segmentation fault 결함을 단독으로 탐지했다. 해당 결함은 탐색 전략, 파라미터 조정 전략, KLEE 기본 설정 모두 도달하지 못한 코드 영역이었으며, 두 전략을 동시에 적용했을 때에만 노출되었다.

4. 결론 및 향후 연구

본 연구는 최신 탐색 전략과 외부 파라미터 조정 전략을 결합하는 세 가지 융합 기법을 시도하였다. 실험 결과, 세 가지 기법 모두 단일 전략이나 KLEE 기본 설정보다 분기 커버리지를 높였으며, 특히 교대 융합은 KLEE 대비 131.8%, 탐색 전략 대비 15.8%, 파라미터 조정 전략 대비 13.6%의 커버리지를 확보해 가장 우수한 성능을 보였다. 아울러 병렬 융합은 combine-0.4.0에서 다른 모든 방법이 놓친 dstring.c:815의 segmentation fault를 단독으로 탐지해, 융합이 커버리지 향상을 넘어 새로운 결함을 발굴할 수 있는 가능성을 시사하였다.

향후 연구는 크게 두 방향으로 확장될 수 있다. 첫째, SMT 풀이 캐시 [4], 경로 가지치기 [5], 상태 병합 [6] 등 기존 자동화 기법을 계층적으로 통합해 커버리지뿐 아니라 SMT 호출 횟수, 메모리 사용량, 테스트 입력 다양성까지 동시에 최적화하는 방안이 있다. 둘째, 실행 과정에서 수집되는 실시간 지표-중간 커버리지 증가율, SMT 대기 시간, 자원 사용률 등-를 활용해 서브 전략의 가중치를 자동으로 조정하고 교대·병렬 모드를 동적으로 전환하는 오케스트레이션을 도입하면, 환경 변화에 적응하면

서 융합 효과를 극대화하는 심볼릭 실행 플랫폼을 구축할 수 있을 것이다.

참고 문헌

- [1] C. Cadar, D. Dunbar, and D. R. Engler, "KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs," Proc. 8th USENIX Symp. on Operating Systems Design and Implementation (OSDI), 2008, pp. 209-224.
- [2] J. Yoon and S. Cha, "FeatMaker: Automated feature engineering for search strategy of symbolic execution," Proc. ACM Softw. Eng., vol. 1, no. FSE, Art. 108, 2024. DOI: 10.1145/3660815.
- [3] S. Cha, M. Lee, S. Lee, and H. Oh, "SymTuner: Maximizing the power of symbolic execution by adaptively tuning external parameters," Proc. 44th Int. Conf. on Software Engineering (ICSE), 2022, pp. 2068-2079. DOI: 10.1145/3510003.3510185.
- [4] Z. Shuai et al., "Partial solution-based constraint solving cache in symbolic execution," Proc. ACM Softw. Eng., vol. 1, no. FSE, Art. 110, 2024. DOI: 10.1145/3660817.
- [5] A. Dutta et al., "TracerX: Pruning dynamic symbolic execution with deletion and weakest-precondition interpolation," Fundam. Approaches Softw. Eng. (FASE), LNCS 14573, 2024, pp. 320-325. DOI: 10.1007/978-3-031-57259-3_19.
- [6] M. Vandercammen and C. De Roover, "State merging for concolic testing of event-driven applications," Sci. Comput. Program., vol. 242, 2025, Art. 103264. DOI: 10.1016/j.scico.2025.103264.